

Object-Oriented Programming in Java

Lecture 10 - Parallel Computing

Emily Lucia Antosch

HAW Hamburg

03.10.2025

Inhaltsverzeichnis

1. Introduction	2
2. Parallel processing	6
3. Class-based threads	13
4. Interface-based threads	19
5. States and selected methods	24
6. Synchronization	35
7. License Notice	41

1. Introduction

1.1 Where are we now?

- In the last lecture, we dealt with output and input
- You can now
 - ▶ send and format output to the console in the right channel,
 - ▶ request input from the user
 - ▶ and read files in Java.
- Today we continue with **parallel computing**.

1.1 Where are we now?

1. Introduction

1. Imperative Konzepte
2. Klassen und Objekte
3. Klassenbibliothek
4. Vererbung
5. Schnittstellen
6. Graphical User Interfaces
7. Exception Handling
8. Input and Output
9. **Multithreading (Parallel Computing)**

1.2 The goal of this chapter

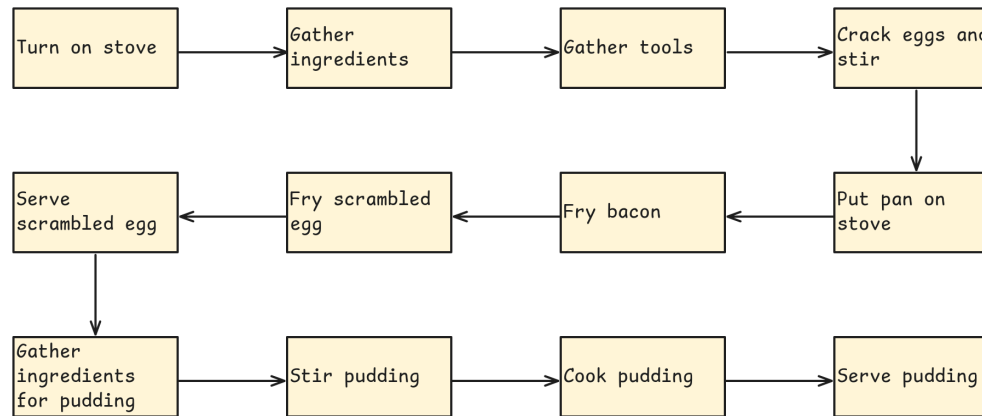
- You execute program code simultaneously in concurrent execution threads (threads).
- You modify the states of active threads to generate the required functionality.
- You synchronize threads and objects to prevent erroneous data states due to incorrect execution orders.

2. Parallel processing

2.1 Scrambled eggs and pudding

2. Parallel processing

- You make scrambled eggs and pudding.
- Possible sequence:



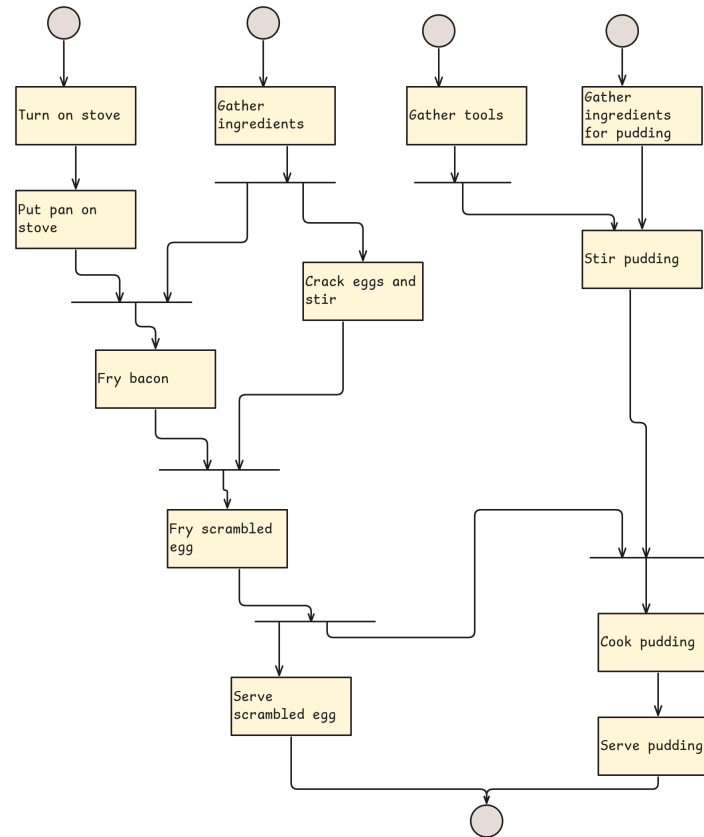
? Frage

- What could the sequence look like if four of you are cooking?
- Constraint: There is only one stove burner

2.1 Scrambled eggs and pudding

2. Parallel processing

- Possible sequence
- Resource conflict: stove burner



2.1 Scrambled eggs and pudding

2. Parallel processing

- Task is divided into subtasks that can be executed in parallel
- Results of subtasks must be exchanged
- Problems:
 - ▶ Dependencies: Subtasks need results of other subtasks
 - ▶ Resource conflict: Subtasks need the same resource
 - ▶ Communication overhead: Exchange of results requires resources and time
- Tasks cannot be parallelized arbitrarily or automatically.

2.1 Scrambled eggs and pudding

2. Parallel processing

- Terms:
 - ▶ Thread (English for „thread“): Execution thread within a program
 - ▶ Multithreading: Multiple (parallel) execution threads within a program
- Memory:
 - ▶ Threads share the memory area of the program:
 - ▶ Therefore share variables and objects
 - ▶ Can communicate efficiently (but unsafely!) via variables and objects
- But: Each thread has its own call stack of called methods

? Frage

- Small riddle in between:
- We have already learned about at least one parallel thread. Which one?

-

-

? Frage

- Small riddle in between:
 - We have already learned about at least one parallel thread. Which one?
-
- Answer:
 - ▶ Garbage Collector (free memory of unreferenced objects)
 - Note:
 - ▶ Java programs create a main thread on startup
 - ▶ Set main() as the bottom method on the call stack
 - ▶ If needed, additionally a thread for the Garbage Collector is started
 - ▶ Program terminates as soon as the last associated thread has terminated

2.1 Scrambled eggs and pudding

2. Parallel processing

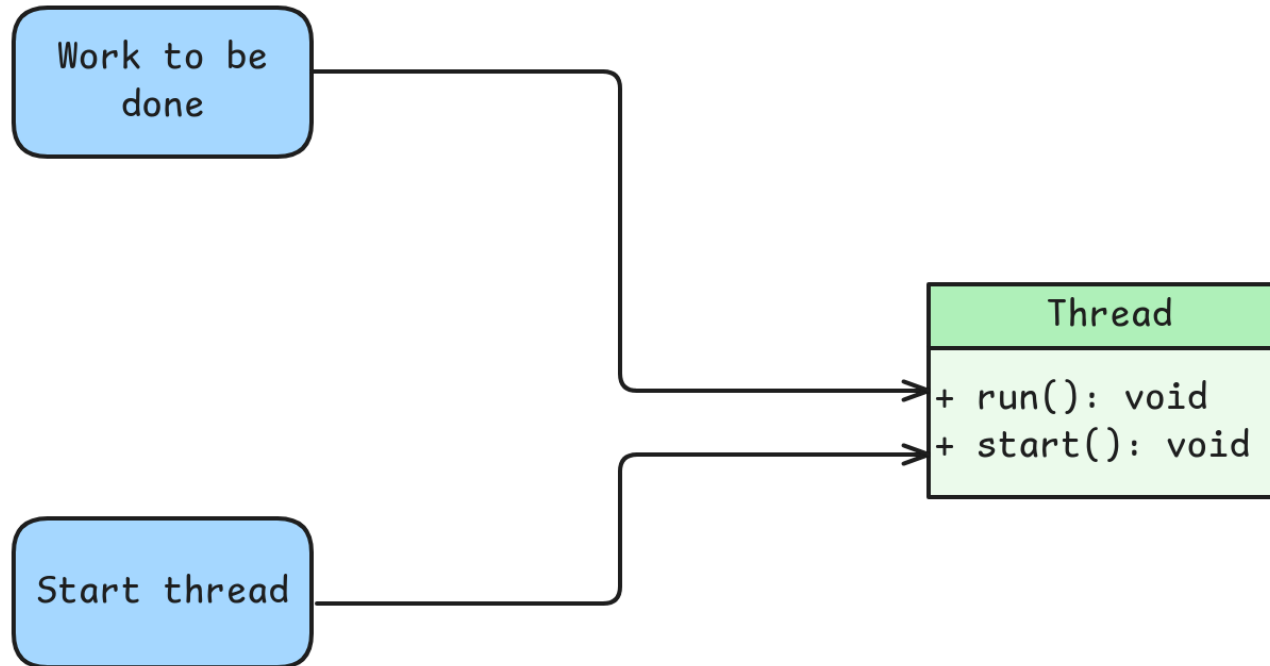
- Allocates computing time (i.e. processors or processor cores) to programs and threads
- Waiting times of other threads or programs are used
- Pseudo-parallelism:
 - ▶ If there are more parallel execution threads than processors or processor cores
 - ▶ Scheduler distributes computing time in slices:
 - Execution in temporal alternation
 - Impression that things are processed in parallel

3. Class-based threads

3.1 Class Thread

3. Class-based threads

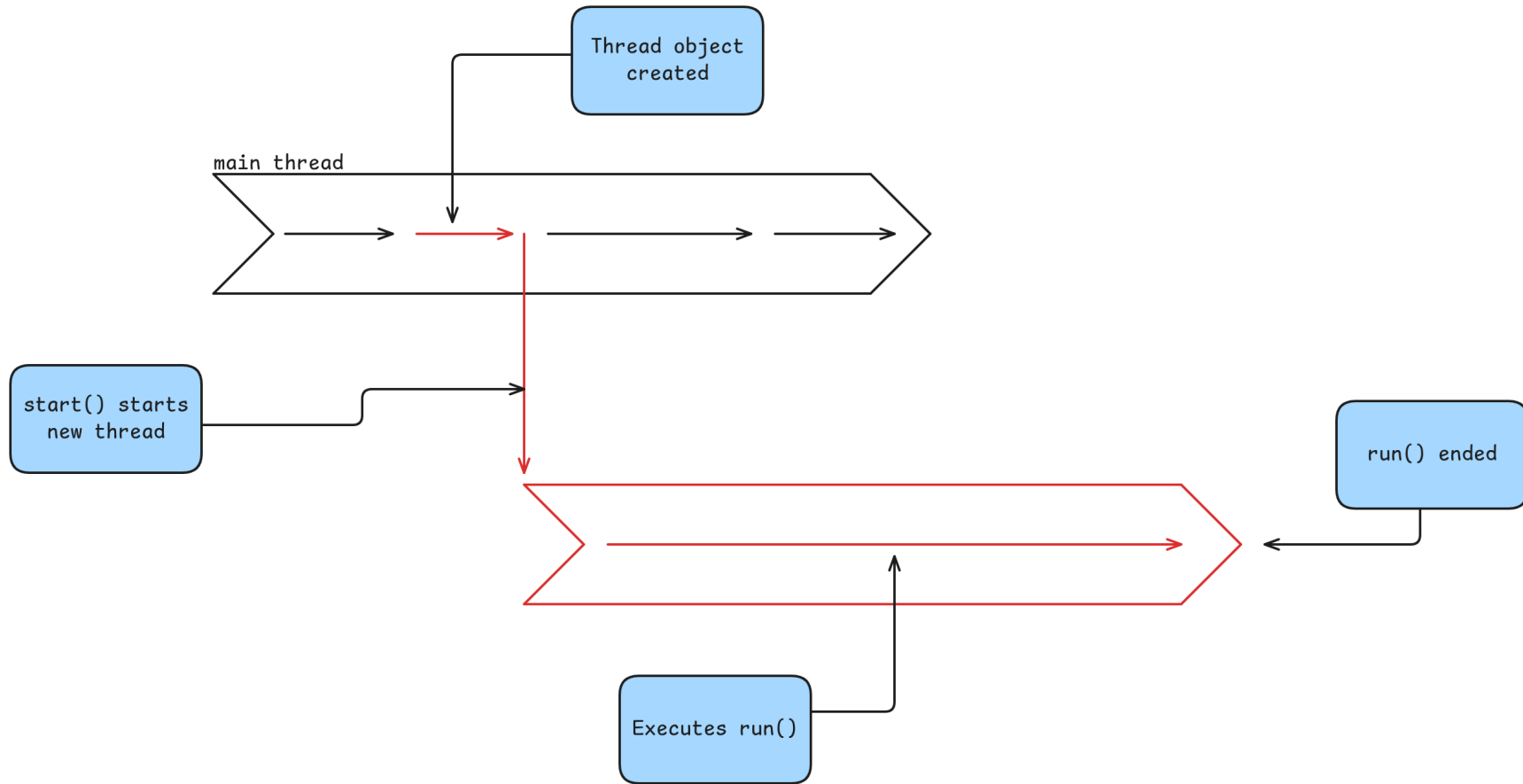
- Threads are created by objects of the Thread class:
- Method `start()` creates and starts parallel execution thread
- Method `run()` contains code to be executed in execution thread
- Execution thread is terminated as soon as `run()` is terminated



3.1 Class Thread

3. Class-based threads

- Illustration



☰ Aufgabe 1

- Let's implement this:
- Write a program that creates an additional thread.

```
1 public class RunThread1 {  
2     public static void main(String[] args) {  
3         Thread thread = new Thread();  
4         System.out.println("Object created");  
5         thread.start();  
6         System.out.println("Thread started");  
7     }  
8 }
```

 Java

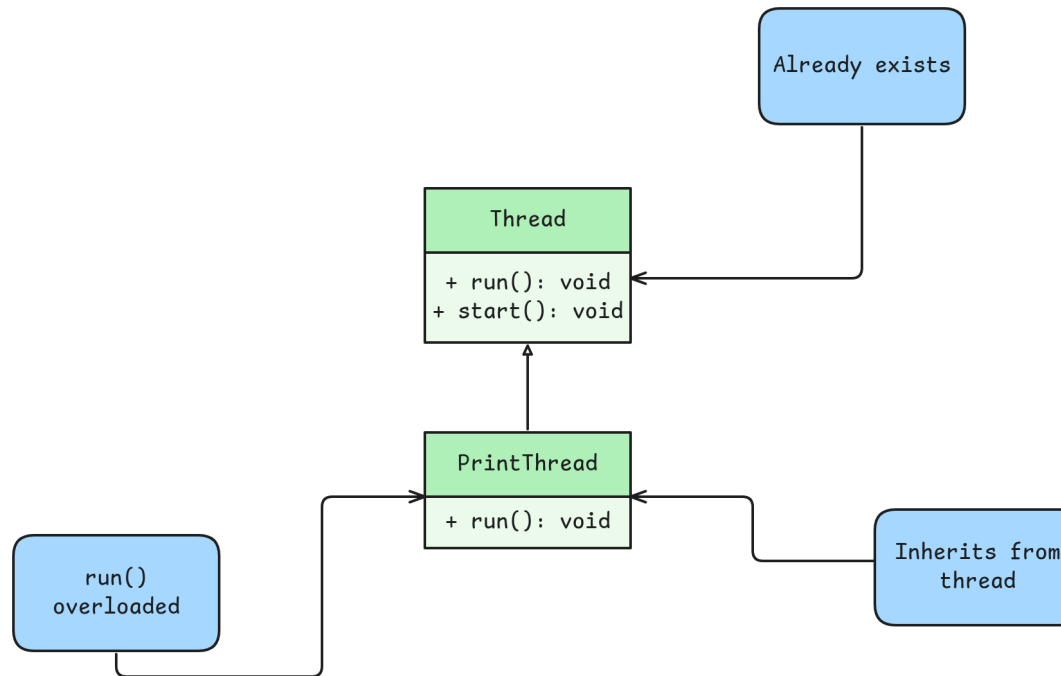
? Frage

- But you can't see anything from the thread!
 - ▶ The run() method of the Thread class is „empty“.
 - ▶ How can we make the thread output text to the console?

3.1 Class Thread

3. Class-based threads

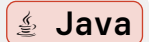
- Approach:
 - ▶ The actual work takes place in the run() method.
 - ▶ The run() method of the Thread class is „empty“.
- Derive your own Thread class from Thread and override run()



☰ Aufgabe 2

- Generate console output in an additional thread.

```
1  public class PrintThread extends Thread {
2      public void run() {
3          System.out.println("Hooray, I'm running in parallel!");
4      }
5  }
6
7  public class RunThread2 {
8      public static void main(String[] args) {
9          PrintThread thread = new PrintThread();
10         System.out.println("Object created");
11         thread.start();
12         System.out.println("Thread started");
13     }
14 }
```

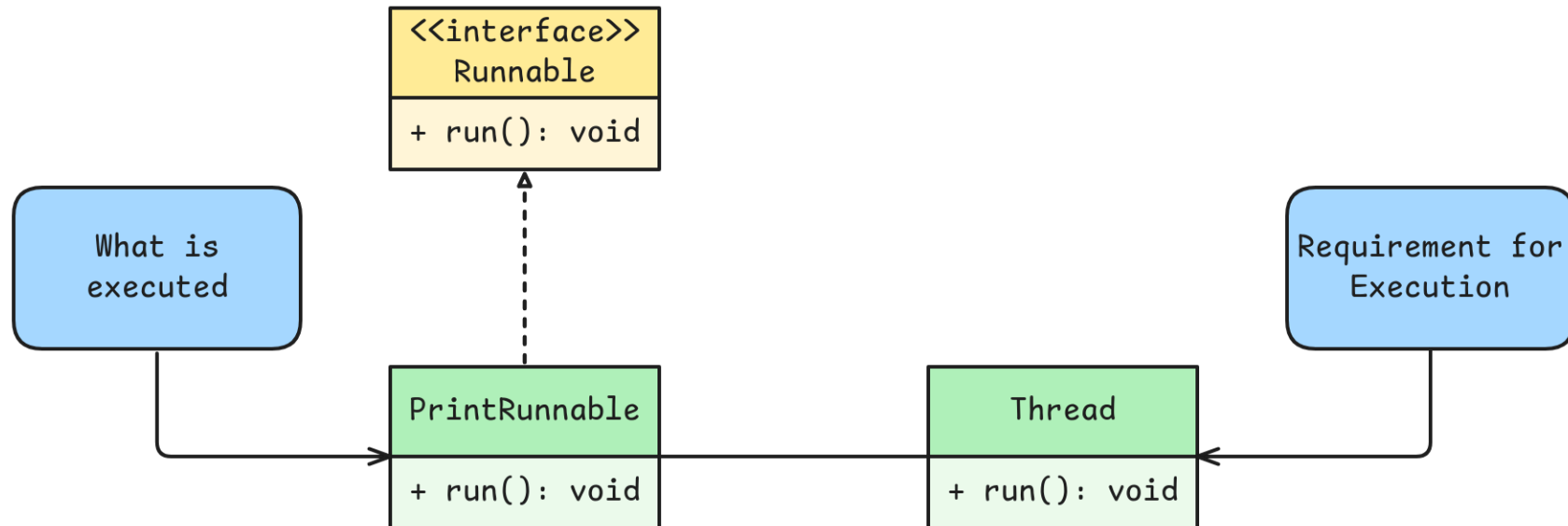


4. Interface-based threads

4.1 Interface Runnable

4. Interface-based threads

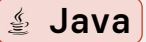
- Alternative to deriving from Thread:
 - ▶ Own class implements interface Runnable with run() method
 - ▶ Runnable object is passed to Thread object: No inheritance required
- Responsibilities:
 - ▶ Runnable object contains what should be executed
 - ▶ Thread object contains everything needed for concurrency



4.1 Interface Runnable

4. Interface-based threads

```
1  public class PrintRunnable implements Runnable {
2      public void run() {
3          System.out.println("Hooray, I'm running in parallel!");
4      }
5  }
6
7  public class InterfaceBased {
8      public static void main(String[] args) {
9          PrintRunnable runnable = new PrintRunnable();
10         Thread thread = new Thread(runnable);
11
12         System.out.println("Objects created");
13         thread.start();
14         System.out.println("Thread started");
15     }
16 }
```



4.1 Interface Runnable

4. Interface-based threads

? Frage

- What is output?

```
1  class CounterRunnable implements Runnable {
2      private int counter;
3      public void run() {
4          while (counter < 10)
5              System.out.println("\t\t\tThread counter: " + counter++);
6              System.out.println("\t\t\tExiting run()");
7      }
8  }
9  public class Counters {
10     private static int counter;
11     public static void main(String[] args) {
12         new Thread(new CounterRunnable()).start();
13         while (counter < 10)
14             System.out.println("Main counter: " + counter++);
15         System.out.println("Exiting main()");
16     }
17 }
```



4.1 Interface Runnable

- Methods `run()` and `main()` count to 9
- Unpredictable who finishes first
- Example output (right):
 - ▶ main thread finished first
 - ▶ Thread with `run()` continues running

4. Interface-based threads

```
Main counter: 0
Main counter: 1
Main counter: 2
Main counter: 3
Main counter: 4
Main counter: 5
Main counter: 6
Main counter: 7
Main counter: 8
Main counter: 9
Exiting main()

Thread counter: 0
Thread counter: 1
Thread counter: 2
Thread counter: 3
Thread counter: 4
Thread counter: 5
Thread counter: 6
Thread counter: 7
Thread counter: 8
Thread counter: 9
Exiting run()
```

main() beendet



5. States and selected methods

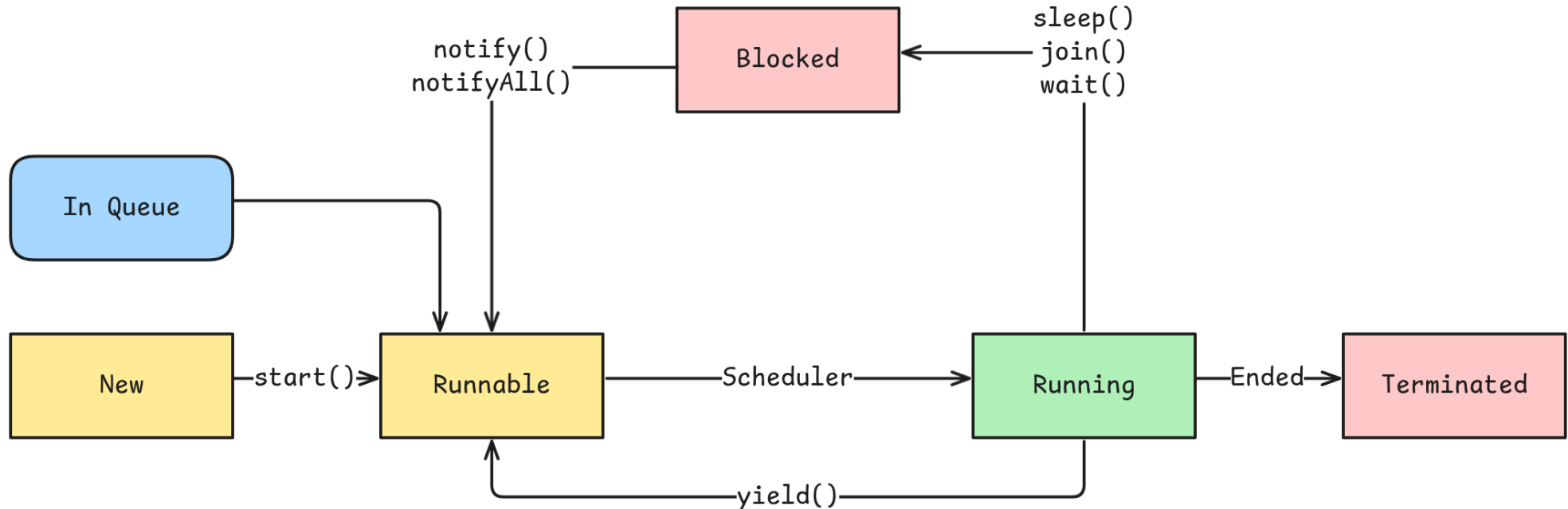
? Frage

- Imagine you were a thread:
 - ▶ What states could you reasonably take?
 - ▶ What state transitions would make sense?
- Don't forget the following:
 - ▶ What happens when there are more threads than processors?
 - ▶ What should you do when waiting for input?

5.1 Thread states

5. States and selected methods

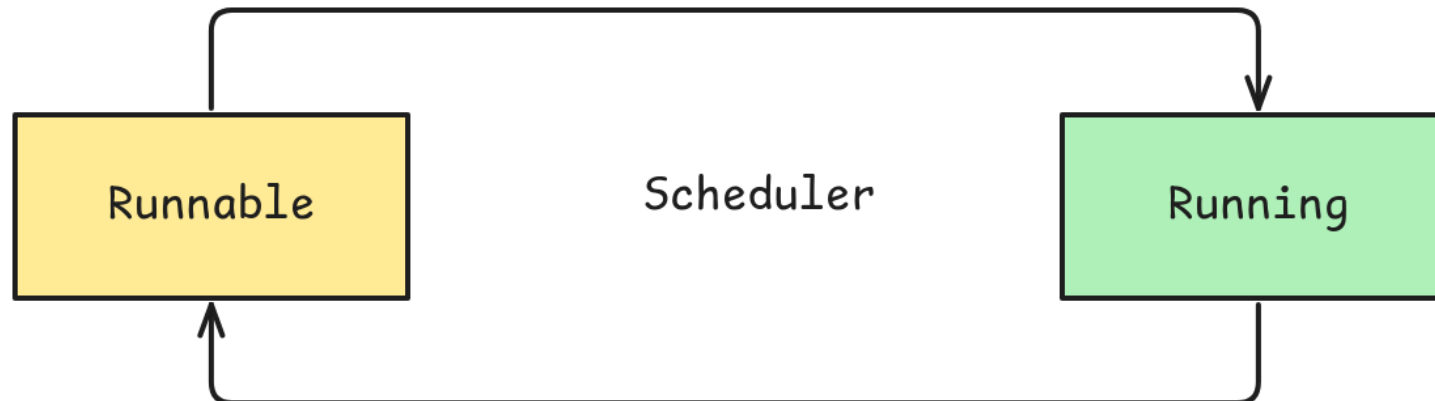
- New: Java object created, but not yet started as a thread
- Runnable: Ready to be executed. Waiting for processor.
- Running: Has processor and is currently being executed
- Blocked: Is not executed and would not be even with a free processor
- Terminated: Thread terminated. Java object still exists!



5.1 Thread states

5. States and selected methods

- Assigns computing time to threads (i.e. Runnable becomes Running)
- Withdraws processor from threads again (i.e. Running becomes Runnable):
 - ▶ Required if more threads than processors exist
 - ▶ Idea: Threads receive computing time alternately
- Control of behavior:
 - ▶ Scheduler is not controllable
 - ▶ No guarantee that threads receive computing time alternately
 - ▶ `setPriority()` sets priority, but no guarantee how scheduler considers it
 - ▶ „The scheduler is a diva!“

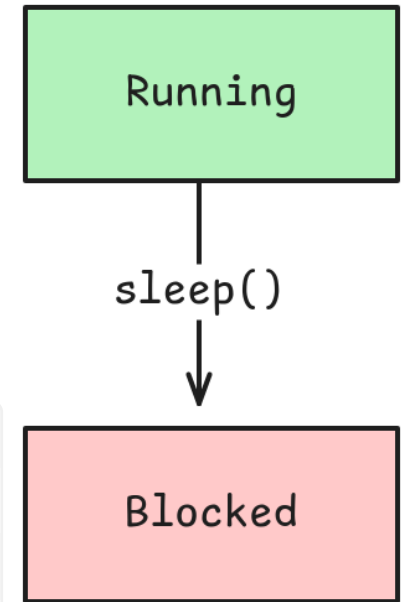


5.1 Thread states

5. States and selected methods

- Put running thread in blocked state for a certain time
- Pass waiting time in milliseconds as parameter (data type long)
- Early wake-up:
 - ▶ Thread can be „woken up“ prematurely by interrupt() method
 - ▶ Throws exception of type InterruptedException

```
1 MyThread thread = new MyThread();  
2 thread.start();  
3 try {  
4     Thread.sleep(1000);  
5 } catch (InterruptedException e) {  
6     e.printStackTrace();  
7 }
```

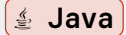


5.1 Thread states

5. States and selected methods

- Make the window blink (every 0.75 s alternating between yellow and light gray):

```
1  public class FlashLight {
2      private boolean isLightOn;
3      private JFrame frame;
4      private FlashLight() {
5          frame = new JFrame("Flashing light");
6          frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
7          frame.setSize(300, 250);
8          frame.setVisible(true);
9      }
10     public void switchLight() {
11         isLightOn = !isLightOn;
12         if (isLightOn)
13             frame.getContentPane().setBackground(Color.YELLOW);
14         else
15             frame.getContentPane().setBackground(Color.LIGHT_GRAY);
16     }
17     public static void main(String[] args) {
18         FlashLight flashLight = new FlashLight();
19     }
20 }
```

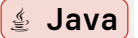


5.1 Thread states

5. States and selected methods

- Required for blinking:
 - ▶ Thread that calls the switchLight() method every 0.75 s

```
1  class FlashThread extends Thread {
2      private FlashLight flashLight;
3
4      public FlashThread(FlashLight flashLight) {
5          this.flashLight = flashLight;
6      }
7
8      public void run() {
9          while (true) {
10             flashLight.switchLight();
11             try {
12                 Thread.sleep(750);
13             } catch (InterruptedException e) {
14             }
15         }
16     }
17 }
```

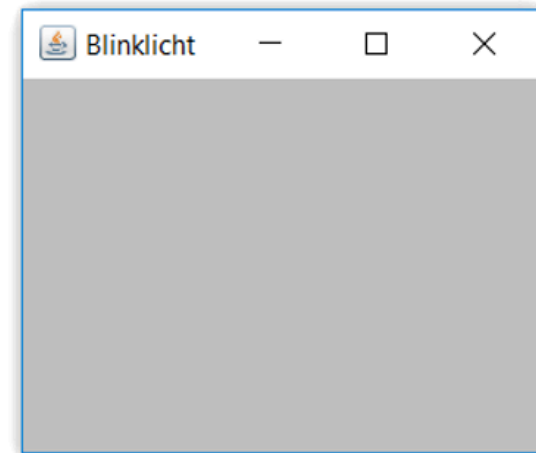
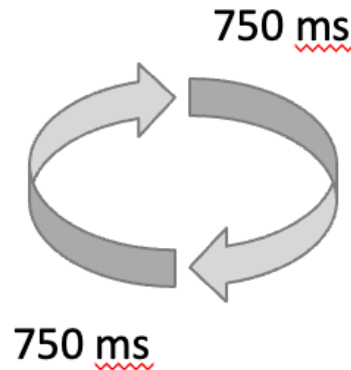
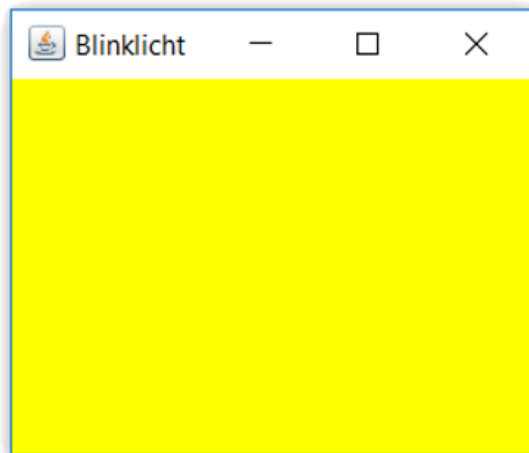
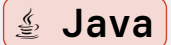


5.1 Thread states

5. States and selected methods

- Creation and starting of the thread in FlashLight:

```
1 public static void main(String[] args) {  
2     FlashLight flashLight = new FlashLight();  
3     FlashThread thread = new FlashThread(flashLight);  
4     thread.start();  
5 }
```

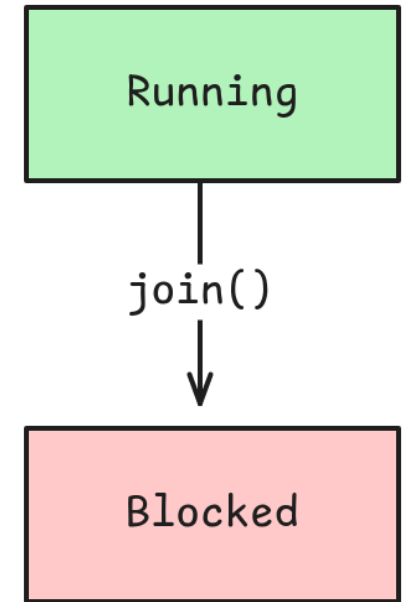


5.1 Thread states

5. States and selected methods

- Make running thread wait for the end of another thread
- Example:
 - ▶ Waits at `thread.join()` until thread terminates
 - ▶ Only then console output occurs

```
1 public static void main(String[] args) {  
2     MyThread thread = new MyThread();  
3     thread.start();  
4     thread.join();  
5     System.out.println("We have joined!");  
6 }
```

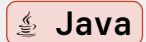


- Maximum waiting time:
 - ▶ Maximum waiting time can be specified as parameter (data type `long`)
 - ▶ What is this needed for? (After all, you don't wait for no reason)

? Frage

- What does this thread do?

```
1  public class SleepyThread extends Thread {
2      public void run() {
3          for (int i = 0; i < 5; i++) {
4              System.out.println("I'm sooo tired ...");
5              try {
6                  Thread.sleep(1000);
7              } catch (InterruptedException e) {
8                  e.printStackTrace();
9              }
10         }
11         System.out.println("Okay, I'm awake again.");
12     }
13 }
```



5.1 Thread states

5. States and selected methods

- It continues:
 - ▶ What output is produced?
 - ▶ What output would be produced without the line `sleepy.join()`?
 - ▶ What output would be produced with `sleepy.join(1500)`?

```
1  public class JoinThreads {
2      public static void main(String[] args) throws InterruptedException {
3          SleepyThread sleepy = new SleepyThread();
4          sleepy.start();
5
6          while (sleepy.isAlive()) {
7              System.out.println("Wake up!");
8              Thread.sleep(400);
9              sleepy.join();
10         }
11         System.out.println("At last ...");
12     }
13 }
```



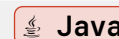
6. Synchronization

6.1 Synchronization

6. Synchronization

- Class represents a bank account with methods for deposits and withdrawals
- Account movements in parallel via threads (e.g. ATM, counter, direct debit)

```
1  public class Account {
2      private double balance;
3
4      public void deposit(double amount) {
5          double newBalance = balance + amount;
6          if (newBalance > balance)
7              balance = newBalance;
8      }
9
10     public void withdraw(double amount) {
11         double newBalance = balance - amount;
12         if (newBalance >= 0.0)
13             balance = newBalance;
14     }
15 }
```



- What happened here?!
 - ▶ You withdraw 50 € while 50 € is credited as a transfer.
 - ▶ Afterwards there are 50 € less than before in the account.

6.1 Synchronization

- Cause:
 - ▶ Threads simultaneously execute methods `deposit()` and `withdraw()`
 - ▶ Both methods access variable `balance`.

Thread 1: <u><code>deposit()</code></u>	Betrag	Thread 2: <u><code>withdraw()</code></u>	Betrag	<u><code>balance</code></u>
<u><code>newBalance = balance + amount;</code></u>	(5050)			5000
		<u><code>newBalance = balance - amount;</code></u>	(4950)	5000
<u><code>if (newBalance > balance)</code></u> <u><code>balance = newBalance;</code></u>				5050
		<u><code>if (newBalance >= 0.0)</code></u> <u><code>balance = newBalance;</code></u>		4950

6.1 Synchronization

- Two threads share a variable.
 - ▶ Race Condition: Result of the program depends on access order
- When does the result depend on which thread is „faster“?
 - ▶ Both threads read the variable
 - ▶ One thread reads, one thread writes to the variable
 - ▶ Both threads write to the variable
- Answer:
 - ▶ Race condition when at least one thread writes

Thread 1	Thread 2	Race Condition
read	read	None, both threads read the same data
read	write	Thread 1 may read value before <i>or</i> after thread 2 writes
write	write	Last-written value remains

Tabelle 1: Formats and Flags

6.1 Synchronization

- Keyword `synchronized` for methods:
 - ▶ Object is locked as soon as a thread enters a synchronized method
 - ▶ Object is released again when thread leaves the method
- Synchronized methods (mutual exclusion):
 - ▶ Object locked: Threads cannot enter synchronized methods. (All synchronized methods are locked, not just the one currently being executed!)
 - ▶ Threads wait in blocked state until the object is released again.
- Non-synchronized methods:
 - ▶ Threads can enter non-synchronized methods when object is locked.

☰ Aufgabe 3

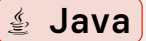
- Help your bank:
 - ▶ Ensure that nothing goes wrong with deposits and withdrawals.

6.1 Synchronization

6. Synchronization

Synchronization via synchronized:

```
1  public class Account {
2      private double balance;
3
4      public synchronized void deposit(double amount) {
5          double newBalance = balance + amount;
6          if (newBalance > balance)
7              balance = newBalance;
8      }
9
10     public synchronized void withdraw(double amount) {
11         double newBalance = balance - amount;
12         if (newBalance >= 0.0)
13             balance = newBalance;
14     }
15 }
```



7. License Notice

7.1 Attribution

- This work is shared under the CC BY-NC-SA 4.0 License and the respective Public License
- `link(„https://creativecommons.org/licenses/by-nc-sa/4.0/“)`
- This work is based off of the work Prof. Dr. Marc Hensel.
- Some of the images and texts, as well as the layout were changed.
- The base material was supplied in private, therefore the link to the source cannot be shared with the audience.